

Rapport de projet — CosmicDeploy

TP final — Industrialisation d'un déploiement applicatif Auteur : Armel Dépôt Git :

<https://github.com/Armel-EFREI/CosmicDeploy> Date : Avril 2026

1. Contexte

Une entreprise souhaite **automatiser le déploiement de son application web** pour gagner du temps et éliminer les erreurs humaines. L'objectif est de construire une chaîne complète allant du code source à une application accessible publiquement, sans aucune action manuelle.

Objectifs

1. Conteneuriser une application (Docker)
2. Orchestrer plusieurs services (Docker Compose)
3. Déployer sur un cluster Kubernetes (K3s) hébergé sur une VM cloud
4. Mettre en place un pipeline CI/CD bloquant sur l'échec des tests
5. Gérer proprement les secrets (aucun identifiant en clair)
6. Documenter le projet

Choix applicatif

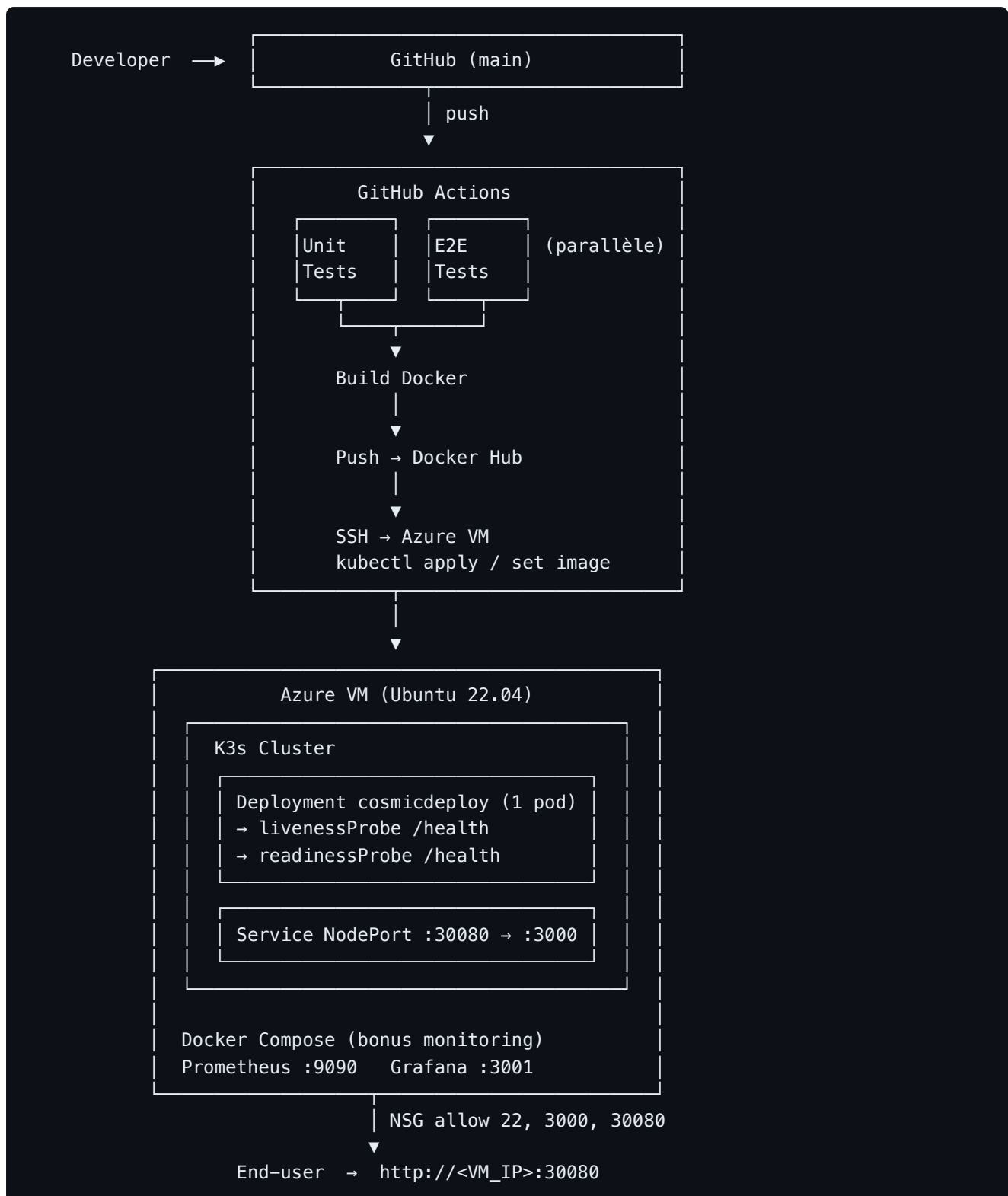
CosmicDeploy — un dashboard CI/CD méta (il affiche l'état de ses propres pipelines). Stack :

- Backend : Node.js 20 + Express
- Frontend : HTML/CSS statique (servi par Express) — design *Cosmic Editorial* (fond sombre, accents ambre)
- Services additionnels : Prometheus + Grafana (bonus monitoring)

Stack de déploiement

Couche	Technologie
Cloud	Azure (VM Standard_B1s , Ubuntu 22.04)
IaC	Terraform
Conteneurisation	Docker (multi-stage)
Orchestration locale	Docker Compose
Orchestration cloud	Kubernetes (K3s)
Registry	Docker Hub
CI/CD	GitHub Actions
Tests	Jest + Supertest
Monitoring	Prometheus + Grafana

2. Architecture



3. Étapes réalisées

Étape 1 — Conteneurisation (Docker)

- Écriture d'un `Dockerfile` **multi-stage** (deps → runtime)
 - Stage 1 : `npm ci --only=production` → image `deps`
 - Stage 2 : copie des `node_modules` prod + du code → image finale minimale
- `HEALTHCHECK` natif Docker sur `/health`
- Image finale : ~120 Mo (Alpine)

Preuve : `docs/screenshots/01-docker-build.png` — sortie de `docker build -t cosmicdeploy .`

Étape 2 — Orchestration Docker Compose

- `docker-compose.yml` orchestre **3 services qui communiquent entre eux** :
 - `app` (port 3002:3000) — expose `/metrics`
 - `prometheus` (port 9090) — scrape `app:3000/metrics`
 - `grafana` (port 3001) — lit les données depuis `prometheus`
- Les services communiquent via le réseau Docker par défaut (résolution DNS par nom de service)

Preuve : `docs/screenshots/02-docker-compose-up.png` + `03-dashboard-local.png`

Étape 3 — Provisioning VM Azure (Terraform — bonus)

- Resource Group + VNet + Subnet
- NSG avec règles : `allow-ssh` (22), `allow-http` (3000), `allow-k8s-nodeport` (30080)
- VM Ubuntu 22.04 `Standard_B1s` avec auth par mot de passe

```
cd terraform
terraform init
terraform apply
# Output : IP publique + commande SSH
```

Preuve : `docs/screenshots/04-terraform-apply.png` + `05-azure-vm-portal.png`

Étape 4 — Installation Docker + K3s sur la VM

```
# Docker
sudo apt update && sudo apt install -y docker.io
sudo usermod -aG docker azureuser

# K3s (distribution Kubernetes légère)
curl -sfL https://get.k3s.io | sh -
sudo chmod 644 /etc/rancher/k3s/k3s.yaml
kubectl get nodes # → VM en Ready
```

Preuve : docs/screenshots/06-k3s-installed.png — `kubectl get nodes`

Étape 5 — Manifests Kubernetes

Création du répertoire `k8s/` contenant :

- `deployment.yaml` — 1 replica, probes liveness/readiness sur `/health`, `imagePullSecrets: regcred`, `resources.requests` + `limits`
- `service.yaml` — `type: NodePort`, `nodePort: 30080` → `targetPort: 3000`
- `kustomization.yaml`

Preuve : docs/screenshots/07-kubectl-apply.png + 08-kubectl-get-all.png

Étape 6 — Pipeline CI/CD GitHub Actions

`.github/workflows/ci-cd.yml` — 4 jobs :

1. **unit-tests** — `npm ci` + `npm run test:unit`
2. **e2e-tests** — `npm ci` + `npm run test:e2e`
3. **build-push** — `needs: [unit-tests, e2e-tests]` — build + push sur Docker Hub (tags `latest` + `<SHA>`)
4. **deploy** — `needs: [build-push]` — SCP des manifests + SSH + `kubectl apply` + `kubectl set image` + `kubectl rollout status` + healthcheck

Contraintes :

-  Pipeline automatique sur `push main`
-  Échec immédiat si un test échoue (les jobs suivants ont `needs`)
-  Déploiement sans aucune action manuelle
-  Aucun secret en clair — tous dans GitHub Secrets

Preuve : docs/screenshots/09-github-actions-success.png + 10-pipeline-stages.png

Étape 7 — Gestion des secrets

Emplacement	Secrets	Usage
GitHub Secrets	<code>DOCKERHUB_USERNAME</code> , <code>DOCKERHUB_TOKEN</code> , <code>AZURE_VM_HOST</code> , <code>AZURE_VM_USER</code> , <code>AZURE_VM_PASSWORD</code>	Pipeline CI/CD
Kubernetes	<code>Secret docker-registry regcred</code>	Pull image Docker Hub depuis K3s
Local dev	<code>.env</code> (dans <code>.gitignore</code>)	Dev uniquement
Exemple committé	<code>.env.example</code>	Documentation

Preuve : docs/screenshots/11-github-secrets.png

Étape 8 — Déploiement en production

Flow complet à partir d'un push :

```
git commit -m "feat: add new feature"
git push origin main
```

→ GitHub Actions enchaîne les 4 jobs (~4 min) :

- Tests unit/E2E OK
- Build + push image `armelefrei/cosmicdeploy:<SHA>`
- SSH sur la VM → `kubectl apply` → rollout du nouveau pod → `curl /health` OK

→ App accessible sur `http://<AZURE_VM_HOST>:30080`

Preuves :

- `docs/screenshots/12-dockerhub-image.png` — image pushée
- `docs/screenshots/13-kubectl-rollout.png` — rollout réussi
- `docs/screenshots/14-app-public-ip.png` — app accessible via IP publique
- `docs/screenshots/15-health-endpoint.png` — `curl http://<IP>:30080/health`

Étape 9 — Bonus monitoring (Prometheus + Grafana)

- Middleware Express qui instrumente chaque requête (4 métriques : `http_requests_total`, `http_response_time_ms`, `http_errors_total`, `memory_usage_mb`)
- Prometheus scrape `/metrics` toutes les 15 s
- Dashboard Grafana pré-provisionné (`monitoring/grafana/dashboards/cosmicdeploy.json`)

Preuve : `docs/screenshots/16-grafana-dashboard.png`

4. Captures d'écran (emplacement)

Toutes les captures sont attendues dans `docs/screenshots/` :

#	Fichier	Contenu
01	01-docker-build.png	<code>docker build</code> — stages et image finale
02	02-docker-compose-up.png	3 services up
03	03-dashboard-local.png	Dashboard en local
04	04-terraform-apply.png	<code>terraform apply</code> — ressources créées
05	05-azure-vm-portal.png	VM visible dans le portail Azure
06	06-k3s-installed.png	<code>kubectl get nodes</code> sur la VM
07	07-kubectl-apply.png	<code>kubectl apply -f k8s/</code>
08	08-kubectl-get-all.png	Deployment + Pod + Service
09	09-github-actions-success.png	Pipeline verte
10	10-pipeline-stages.png	4 jobs détaillés
11	11-github-secrets.png	Liste des secrets configurés
12	12-dockerhub-image.png	Tags <code>latest</code> + <code><SHA></code> sur Docker Hub
13	13-kubectl-rollout.png	<code>rollout status</code> OK
14	14-app-public-ip.png	App accessible sur <code>http://<IP>:30080</code>
15	15-health-endpoint.png	<code>/health</code> retourne <code>{ status: "ok", ... }</code>
16	16-grafana-dashboard.png	Dashboard Grafana avec métriques

5. Difficultés rencontrées et résolutions

5.1. Kubeconfig K3s inaccessible en mode non-root

Symptôme : dans le script SSH de la pipeline, `kubectl` retournait *permission denied* sur `/etc/rancher/k3s/k3s.yaml`. **Cause** : K3s écrit le kubeconfig en mode `600` root-only. **Résolution** : ajout de `sudo chmod 644 /etc/rancher/k3s/k3s.yaml + export KUBECONFIG=...` dans le script SSH.

5.2. K3s n'arrive pas à pull l'image depuis Docker Hub

Symptôme : pod en `ImagePullBackOff` au premier déploiement. **Cause** : K3s utilise containerd (pas Docker). Un `docker login` / `docker pull` sur la VM n'aide pas : c'est containerd qui pull, via sa propre config. **Résolution** : création d'un `Secret docker-registry regcred` (idempotent via `--dry-run=client -o yaml` | `kubectl apply -f -`) référencé par `imagePullSecrets` dans le `Deployment`.

5.3. NodePort inaccessible depuis Internet

Symptôme : `curl http://<VM_IP>:30080/health` → timeout depuis une machine externe. **Cause** : le NSG Azure ne laissait passer que le port 3000. **Résolution** : ajout d'une règle `allow-k8s-nodeport` (priorité 120, port 30080) dans `terraform/main.tf`.

5.4. `kubectl apply` ne redémarre pas le pod

Symptôme : après un nouveau push, le pod ne redémarrait pas même si le SHA avait changé. **Cause** : quand le YAML a exactement la même image string, `apply` dit *unchanged* et ne trigge pas de rollout. **Résolution** : après `apply`, un `kubectl set image deployment/cosmicdeploy cosmicdeploy=<IMAGE>:<SHA>` force systématiquement la mise à jour.

5.5. Healthcheck déclenché avant que le pod soit prêt

Symptôme : `curl /health` échouait, la pipeline était rouge, mais le pod était en cours de démarrage. **Cause** : pas d'attente de readiness. **Résolution** : `kubectl rollout status deployment/cosmicdeploy --timeout=180s` avant le `curl`. Si le pod ne devient pas `Ready` en 3 min, la pipeline échoue proprement avec les logs.

5.6. Warnings de dépréciation Node 16 sur les actions GitHub

Symptôme : `appleboy/ssh-action` émettait *Node.js 16 actions are deprecated*. **Résolution** : ajout de `FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true` dans `env:` au niveau du workflow.

6. Résultats

Contraintes TP — checklist

Exigence	Statut
Application fonctionne en local	✓
Expose <code>/health</code>	✓
Services communiquent entre eux (app ↔ Prometheus ↔ Grafana)	✓
Dockerfile par service	✓ (app ; Prometheus/Grafana = images officielles)
<code>docker-compose up</code> lance tout	✓
Ports exposés correctement	✓ (3002, 9090, 3001)
VM cloud créée	✓ (Azure, via Terraform)
Docker + Docker Compose installés sur la VM	✓
Kubernetes (K3s) sur la VM	✓
Application déployée dans K8s	✓
App accessible via IP de la VM	✓ (<code>http://<IP>:30080</code>)
Pipeline GitHub Actions complet	✓ (4 jobs)
Pipeline automatique (push → déploiement)	✓
Pipeline échoue si tests échouent	✓ (<code>needs:</code> entre jobs)
Déploiement sans action manuelle	✓
Secrets jamais en clair dans le code	✓
README (description, pipeline, étapes, difficultés)	✓

Bonus

Bonus	Statut
Multi-stage Dockerfile	✓
Terraform pour la VM	✓
Prometheus + Grafana	✓
Cluster K8s managé (AKS/GKE)	✗ (K3s local sur VM à la place, conforme au TP)

7. Conclusion

La pipeline **fonctionne de bout en bout** : un simple `git push` déclenche tests → build → push Docker Hub → déploiement K8s → healthcheck. Le déploiement est **idempotent** (K8s gère le rolling update) et **bloquant sur l'échec** (tests, rollout, healthcheck).

Le choix de **K3s** plutôt que Minikube s'est avéré pertinent pour une VM `Standard_B1s` (1 vCPU / 1 Go RAM) : K3s tourne en moins de 150 Mo de RAM et démarre en quelques secondes.

Les bonus (Terraform + monitoring) montrent une chaîne de déploiement réaliste qu'on retrouverait en entreprise : infrastructure versionnée, observabilité intégrée, secrets centralisés.

Améliorations possibles :

- Migrer vers un cluster managé (AKS) avec `LoadBalancer` Azure → IP dédiée + TLS
- Remplacer l'auth SSH par mot de passe par une clé privée (plus sécurisé)
- Ajouter Helm pour le packaging des manifests
- Ajouter un `HorizontalPodAutoscaler` basé sur les métriques Prometheus

Lien du dépôt Git : <https://github.com/Armel-EFREI/CosmicDeploy>